7.3 Doubly Linked Lists

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list. In the opening of Section 7.1, we emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, but we are unable to efficiently delete a node at the tail of the list. More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately *precedes* the node to be deleted (yet, that node needs to have its next reference updated).

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a ***doubly linked list***. These lists allow a greater variety of $O(1)$-time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term "next" for the reference to the node that follows another, and we introduce the term "prev" for the reference to the node that precedes it.

Header and Trailer Sentinels

In order to avoid some special cases when operating near the boundaries of a doubly linked list, it helps to add special nodes at both ends of the list: a ***header*** node at the beginning of the list, and a ***trailer*** node at the end of the list. These "dummy" nodes are known as ***sentinels*** (or guards), and they do not store elements of the primary sequence. A doubly linked list with such sentinels is shown in Figure 7.10.

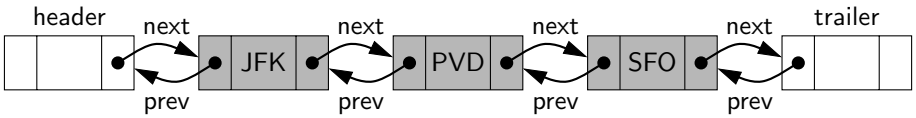


Figure 7.10: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

When using sentinel nodes, an empty list is initialized so that the next field of the header points to the trailer, and the prev field of the trailer points to the header; the remaining fields of the sentinels are irrelevant (presumably None, in Python). For a nonempty list, the header's next will refer to a node containing the first real element of a sequence, just as the trailer's prev references the node containing the last element of a sequence.

Advantage of Using Sentinels

Although we could implement a doubly linked list without sentinel nodes (as we did with our singly linked list in Section 7.1), the slight extra space devoted to the sentinels greatly simplifies the logic of our operations. Most notably, the header and trailer nodes never change—only the nodes between them change. Furthermore, we can treat all insertions in a unified manner, because a new node will always be placed between a pair of existing nodes. In similar fashion, every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side.

For contrast, look back at our LinkedQueue implementation from Section 7.1.2. Its enqueue method, given in Code Fragment 7.8, adds a new node to the end of the list. However, its implementation required a conditional to manage the special case of inserting into an empty list. In the general case, the new node was linked after the existing tail. But when adding to an empty list, there is no existing tail; instead it is necessary to reassign self._head to reference the new node. The use of a sentinel node in that implementation would eliminate the special case, as there would always be an existing node (possibly the header) before a new node.

Inserting and Deleting with a Doubly Linked List

Every insertion into our doubly linked list representation will take place between a pair of existing nodes, as diagrammed in Figure 7.11. For example, when a new element is inserted at the front of the sequence, we will simply add the new node *between* the header and the node that is currently after the header. (See Figure 7.12.)

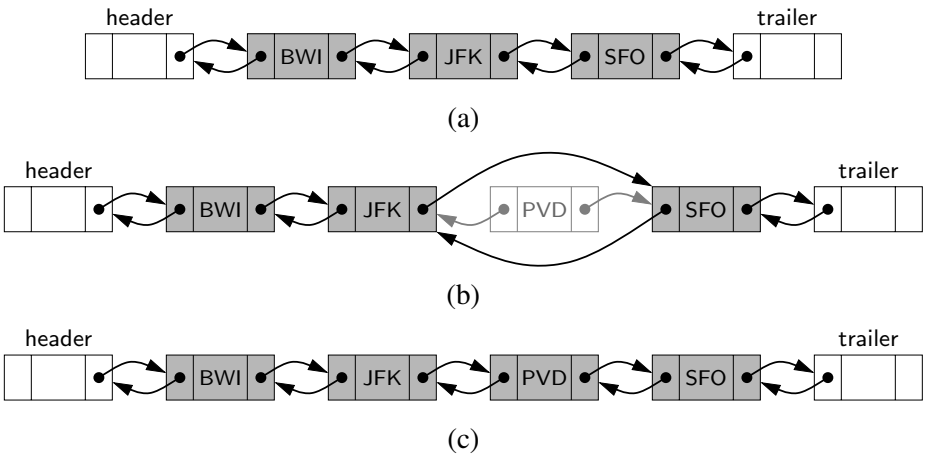


Figure 7.11: Adding an element to a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

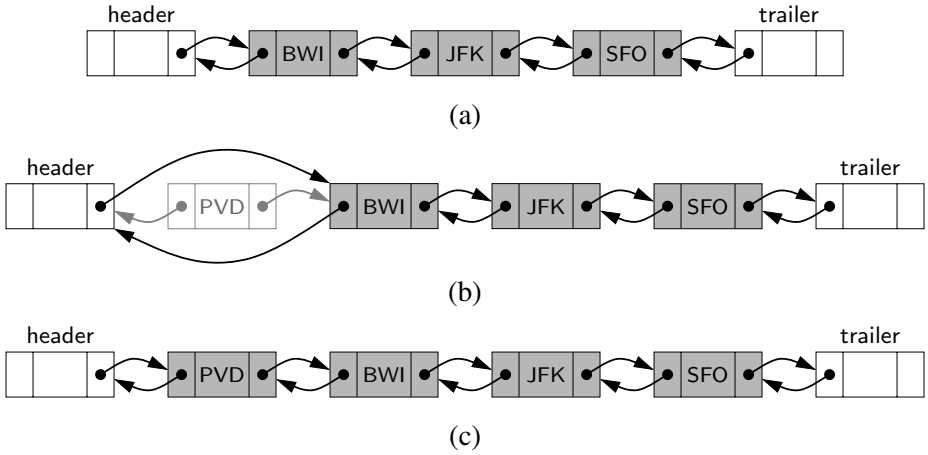


Figure 7.12: Adding an element to the front of a sequence represented by a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

The deletion of a node, portrayed in Figure 7.13, proceeds in the opposite fashion of an insertion. The two neighbors of the node to be deleted are linked directly to each other, thereby bypassing the original node. As a result, that node will no longer be considered part of the list and it can be reclaimed by the system. Because of our use of sentinels, the same implementation can be used when deleting the first or the last element of a sequence, because even such an element will be stored at a node that lies between two others.

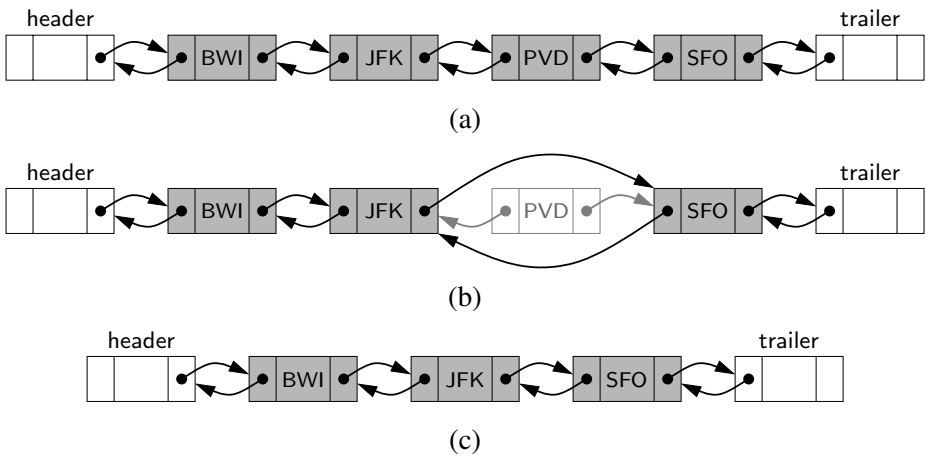


Figure 7.13: Removing the element PVD from a doubly linked list: (a) before the removal; (b) after linking out the old node; (c) after the removal (and garbage collection).

7.3.1 Basic Implementation of a Doubly Linked List

We begin by providing a preliminary implementation of a doubly linked list, in the form of a class named _DoublyLinkedBase. We intentionally name the class with a leading underscore because we do not intend for it to provide a coherent public interface for general use. We will see that linked lists can support general insertions and deletions in $O(1)$ worst-case time, but only if the location of an operation can be succinctly identified. With array-based sequences, an integer index was a convenient means for describing a position within a sequence. However, an index is not convenient for linked lists as there is no efficient way to find the jth element; it would seem to require a traversal of a portion of the list.

When working with a linked list, the most direct way to describe the location of an operation is by identifying a relevant node of the list. However, we prefer to encapsulate the inner workings of our data structure to avoid having users directly access nodes of a list. In the remainder of this chapter, we will develop two public classes that inherit from our _DoublyLinkedBase class to provide more coherent abstractions. Specifically, in Section 7.3.2, we provide a LinkedDeque class that implements the double-ended queue ADT introduced in Section 6.3; that class only supports operations at the ends of the queue, so there is no need for a user to identify an interior position within the list. In Section 7.4, we introduce a new PositionalList abstraction that provides a public interface that allows arbitrary insertions and deletions from a list.

Our low-level _DoublyLinkedBase class relies on the use of a nonpublic _Node class that is similar to that for a singly linked list, as given in Code Fragment 7.4, except that the doubly linked version includes a _prev attribute, in addition to the _next and _element attributes, as shown in Code Fragment 7.11.

```
class _Node:
  """Lightweight, nonpublic class for storing a doubly linked node."""
  __slots__ = '_element', '_prev', '_next'  # streamline memory

  def __init__(self, element, prev, next):          # initialize node's fields
    self._element = element                         # user's element
    self._prev = prev                               # previous node reference
    self._next = next                               # next node reference
```

Code Fragment 7.11: A Python _Node class for use in a doubly linked list.

The remainder of our _DoublyLinkedBase class is given in Code Fragment 7.12. The constructor instantiates the two sentinel nodes and links them directly to each other. We maintain a _size member and provide public support for __len__ and is_empty so that these behaviors can be directly inherited by the subclasses.

```
class _DoublyLinkedBase:
  """A base class providing a doubly linked list representation."""

  class _Node:
    """Lightweight, nonpublic class for storing a doubly linked node."""
    (omitted here; see previous code fragment)

  def __init__(self):
    """Create an empty list."""
    self._header = self._Node(None, None, None)
    self._trailer = self._Node(None, None, None)
    self._header._next = self._trailer                # trailer is after header
    self._trailer._prev = self._header                # header is before trailer
    self._size = 0                                    # number of elements

  def __len__(self):
    """Return the number of elements in the list."""
    return self._size

  def is_empty(self):
    """Return True if list is empty."""
    return self._size == 0

  def _insert_between(self, e, predecessor, successor):
    """Add element e between two existing nodes and return new node."""
    newest = self._Node(e, predecessor, successor)   # linked to neighbors
    predecessor._next = newest
    successor._prev = newest
    self._size += 1
    return newest

  def _delete_node(self, node):
    """Delete nonsentinel node from the list and return its element."""
    predecessor = node._prev
    successor = node._next
    predecessor._next = successor
    successor._prev = predecessor
    self._size -= 1
    element = node._element                           # record deleted element
    node._prev = node._next = node._element = None    # deprecate node
    return element                                    # return deleted element
```

Code Fragment 7.12: A base class for managing a doubly linked list.

The other two methods of our class are the nonpublic utilities, _insert_between and _delete_node. These provide generic support for insertions and deletions, respectively, but require one or more node references as parameters. The implementation of the _insert_between method is modeled upon the algorithm that was previously portrayed in Figure 7.11. It creates a new node, with that node's fields initialized to link to the specified neighboring nodes. Then the fields of the neighboring nodes are updated to include the newest node in the list. For later convenience, the method returns a reference to the newly created node.

The implementation of the _delete_node method is modeled upon the algorithm portrayed in Figure 7.13. The neighbors of the node to be deleted are linked directly to each other, thereby bypassing the deleted node from the list. As a formality, we intentionally reset the _prev, _next, and _element fields of the deleted node to None (after recording the element to be returned). Although the deleted node will be ignored by the rest of the list, setting its fields to None is advantageous as it may help Python's garbage collection, since unnecessary links to the other nodes and the stored element are eliminated. We will also rely on this configuration to recognize a node as "deprecated" when it is no longer part of the list.

7.3.2 Implementing a Deque with a Doubly Linked List

The double-ended queue (deque) ADT was introduced in Section 6.3. With an array-based implementation, we achieve all operations in *amortized* $O(1)$ time, due to the occasional need to resize the array. With an implementation based upon a doubly linked list, we can achieve all deque operation in *worst-case* $O(1)$ time.

We provide an implementation of a LinkedDeque class (Code Fragment 7.13) that inherits from the _DoublyLinkedBase class of the preceding section. We do not provide an explicit __init__ method for the LinkedDeque class, as the inherited version of that method suffices to initialize a new instance. We also rely on the inherited methods __len__ and is_empty in meeting the deque ADT.

With the use of sentinels, the key to our implementation is to remember that the header does not store the first element of the deque—it is the node just *after* the header that stores the first element (assuming the deque is nonempty). Similarly, the node just *before* the trailer stores the last element of the deque.

We use the inherited _insert_between method to insert at either end of the deque. To insert an element at the front of the deque, we place it immediately between the header and the node just after the header. An insertion at the end of deque is placed immediately before the trailer node. Note that these operations succeed, even when the deque is empty; in such a situation, the new node is placed between the two sentinels. When deleting an element from a nonempty deque, we rely upon the inherited _delete_node method, knowing that the designated node is assured to have neighbors on each side.

```
1  class LinkedDeque(_DoublyLinkedBase):          # note the use of inheritance
2    """Double-ended queue implementation based on a doubly linked list."""
3
4    def first(self):
5      """Return (but do not remove) the element at the front of the deque."""
6      if self.is_empty( ):
7        raise Empty("Deque is empty")
8      return self._header._next._element           # real item just after header
9
10   def last(self):
11     """Return (but do not remove) the element at the back of the deque."""
12     if self.is_empty( ):
13       raise Empty("Deque is empty")
14     return self._trailer._prev._element          # real item just before trailer
15
16   def insert_first(self, e):
17     """Add an element to the front of the deque."""
18     self._insert_between(e, self._header, self._header._next)   # after header
19
20   def insert_last(self, e):
21     """Add an element to the back of the deque."""
22     self._insert_between(e, self._trailer._prev, self._trailer)    # before trailer
23
24   def delete_first(self):
25     """Remove and return the element from the front of the deque.
26
27     Raise Empty exception if the deque is empty.
28     """
29     if self.is_empty( ):
30       raise Empty("Deque is empty")
31     return self._delete_node(self._header._next)       # use inherited method
32
33   def delete_last(self):
34     """Remove and return the element from the back of the deque.
35
36     Raise Empty exception if the deque is empty.
37     """
38     if self.is_empty( ):
39       raise Empty("Deque is empty")
40     return self._delete_node(self._trailer._prev)      # use inherited method
```

Code Fragment 7.13: Implementation of a LinkedDeque class that inherits from the _DoublyLinkedBase class.